Immune Evolutionary Generative Adversarial Networks for Hyperspectral Image Classification

Jing Bai, *Senior Member, IEEE*, Yang Zhang, Zhu Xiao, *Senior Member, IEEE*, Fawang Ye, You Li, Mamoun Alazab, *Senior Member, IEEE*, and Licheng Jiao, *Fellow, IEEE*

***Abstract*— In recent years, hyperspectral image classification (HIC) algorithm based on deep learning has been widely studied, and has achieved much better results than traditional algorithms. HIC using small samples has gradually become a research hotspot, and the generative adversarial networks (GANs) have become a brilliant application in this field. However, the HIC results based on GAN methods are poor and volatile, since a single loss function cannot accurately measure the distance between the generated samples and the real samples in different hyperspectral images. To resolve this problem, we propose a novel immune evolutionary generative adversarial network (HIEGAN) via leveraging the evolutionary strategy and immune strategy. Specifically, we enhance the performance of the generator in two ways: 1) HIEGAN uses multiple loss functions for calculation and backpropagation, so as to endow the generator with different parameter values and select the best one as the evolution result each time to enter the next iteration and 2) in the training process, we preserve the optimal generator as memory cells to avoid the performance degradation of the generator. Through these changes, HIEGAN overcame the defects of GAN, improved the stability of GAN, and finally improved classification efficiency. At the same time, in order to alleviate the overfitting problem of depth network under small samples, we change convolution and deconvolution into ghost module to reduce the network parameters. Experiments on three classical datasets validate that HIEGAN has encouraging performance in HIC under small samples.**

***Index Terms*— Generative adversarial networks (GANs), hyperspectral image classification (HIC), immune evolutionary algorithm.**

Manuscript received 18 March 2022; revised 30 June 2022; accepted 17 September 2022. Date of publication 11 November 2022; date of current version 28 November 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62276206; in part by the Fundamental Research Funds for the Central Universities under Grant QTZX22088; in part by the Key Research and Development Program of Shaanxi under Grant S2022-YF-YBGY-0921 and Grant 2020GXLHY023; in part by the Key Research and Development Project of Hunan Province of China under Grant 2022GK2020; in part by the Science and Technology Project of Hunan Provincial Water Resources Department under Grant XSKJ2021000-39; in part by the Project of SKL of Remote Sensing Information and Image Analysis Technology under Grant 6142A010409; in part by the Open Research Fund from the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ) under Grant GML-KF-22-22; and in part by the Open Research Fund Program of the Key Laboratory of Digital Mapping and Land Information Application, Ministry of Natural Resources, under Grant ZRZYBWD202205. *(Corresponding author: Zhu Xiao.)*

Jing Bai and Licheng Jiao are with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, School of Artificial Intelligence, Xidian University, Xi'an 710071, China (e-mail: baijingmail.xidian.edu.cn; lchjiao@mail.xidian.edu.cn).

Yang Zhang is with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, School of Artificial Intelligence, Xidian University, Xi'an 710071, China, and also with Key Laboratory of Digital Mapping and Land Information Application, School of Resource and Environmental Sciences, Wuhan University, Wuhan 430079, China (e-mail: yangzhang_8@stu.xidian.edu.cn).

Zhu Xiao is with the College of Computer Science and Electronics Engineering, Hunan University, Changsha 410082, China (e-mail: zhxiao@hnu.edu.cn).

Fawang Ye is with the National Key Laboratory of Remote Sensing Information and Imagery Analysis, Beijing Research Institute of Uranium Geology, Beijing 100029, China (e-mail: yfwbeijing2008@sina.com).

You Li is with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen 518060, China (e-mail: liyougis@gmail.com).

Mamoun Alazab is with the College of Engineering, IT and Environment, Charles Darwin University, Darwin, NT 0810, Australia (e-mail: mamoun.alazab@cdu.edu.au).

Digital Object Identifier 10.1109/TGRS.2022.3210280

I. Introduction

HYPERSPECTRAL imaging technology is an important milestone in the development of remote sensing field. Through the hyperspectral sensors mounted on different space platforms, namely imaging spectrometer, the target area can be imaged simultaneously in hundreds of continuous and subdivided spectral bands [1]. In this way, hyperspectral image (HSI) contains both spectral and spatial information of the target region, which is rich in information, large amount of data, and strong recognition ability. HSI is widely used in military, geography, agriculture, aerospace, and other fields, wherein HSI classification is a necessary step in the application of hyperspectral technology.

HSIs contain multiple bands data, and the excessive increase of features leads to "Hughes phenomenon." Therefore, researchers all over the world have designed many methods for feature extraction and dimensionality reduction, such as principal component analysis (PCA) [2], independent component analysis (ICA) [3], linear discriminant analysis (LDA) [4], band selection (BS) [5], and so on. In the early days of hyperspectral image classification (HIC), researchers mainly used spectral features of pixels for classification, such as neural network [6], maximum likelihood classification [7], polynomial logistic regression [8], [9], and sparse representation classification [10]. Because of not considering the spatial background, the results of these pixel-based methods are not satisfactory, and the classification methods based on spectral–spatial information gradually appear and become the mainstream. Among them, the classical methods are composite kernel function method [11], extended morphological contour method (EMP) [12], differential morphological contour method (DMP) [13], joint sparse representation and Markov random field (MRF) [14], edge preserving filtering method (EPF) [15], and extreme learning machine method [16]. Before deep learning became mainstream, the support vector machine (SVM) [17] series of classifiers showed good classification results. Many variants of SVM models for HSI classification

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

were proposed, such as Markov SVM (SVM-MRF) [18], SVM based on spatial morphological contour (MPS-SVM) [19], and Laplace SVM [20]. The depth stacking of single structure is considered to be effective. In [21], Liu et al. extracted the morphological attribute contour cube of HSI as the feature of the sample, and classified it with depth random forest (MAPC-DRF) with remarkable effect.

In recent years, deep learning has been widely used in HSI image classification. The deep learning automatically extracts the HSI features through the multilayer network structure and classifies them directly. Deep learning can not only classify the end-to-end, but also greatly improve the classification results compared with traditional methods. In 2014, Chen et al. [22] first used deep learning in HSI classification via combing PCA, deep learning architecture, and logic regression, and proposed a stack automatic encoder (SAE) framework for HSI classification. Li et al. [23] used limited Boltzmann machine and depth confidence network (DBN) to classify HSI, and proposed the corresponding HSI classification framework DBN. DBN first pretrained the unlabeled samples without supervision, and then adjusted the labeled samples with supervision. SAE and DBN allow unsupervised training with high classification accuracy. However, the two network layers are completely connected, and convolutional network (CNN) is popular because of its shared weight and reduced parameters. CNN is able to extract more effective features. There are many different structures for classification using CNN [24], [25], [26]. HSI is 3-D. Chen et al. [27] used three-dimensional convolutional neural network (3DCNN) to classify HSI, convoluted 3-D image with 3-D convolution kernel directly, and obtained competitive results, but the use of 3-D convolution kernel also greatly increased the amount of calculation.

Compared with the previous methods, CNN series methods achieve better classification results. However, neural network contains a great quantity of parameters, and training these parameters requires a lot of labeled data. In the field of hyperspectral, labeled samples are scarce. To solve this issue, researchers have tried to generate new HSI data based on the original training data, which mainly includes two strategies. One is traditional, which directly recombines and relabels the existing HSI to obtain new samples. For example, Yu et al. [28] conducted HSI data augmentation by rotating and flipping the original data. Li et al. [29] use pixels to classify features [CNN-deep pixel-pair features (PPF)]. The other is to generate data by generative adversarial networks (GANs) [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], which is composed of a generator that aims to generate hyperspectral samples and a discriminator that aims to distinguish the authenticity of hyperspectral samples and classify them. In the continuous training, the two structures are optimized with the adversarial objective alternately, and finally both parts obtain excellent performance. GAN can be used to produce HSI samples and then be classified by other classifiers [30], or it can be directly classified end-to-end.

At present, there are many GAN methods used in HSI classification task. Zhu et al. [39] first applied GAN to HSI classification, and proposed 1D-GAN as a spectral classifier and 3D-GAN as a spectral–spatial classifier. In [31], Zhan et al. proposed a model of semisupervised GANs for HIC, called HSGAN. The model uses all unlabeled data to fit a 1-D GAN structure, then replaces the last layer of GAN discriminator with softmax layer, and fine tune with a small number of samples to classify. Zhong et al. [32] combined conditional random fields (CRF) with GAN and proposed GAN-CRF framework for semisupervised small sample classification. Zhang et al. [38] used Wasserstein distance instead of the original Jensen–Shannon (JS) divergence, namely WGAN, to train HSI images end-to-end unsupervised. Feng et al. [33] proposed multiclass spectral–spatial GAN (MSGAN) to extract spectral information and spatial information by using dual generators. In [34], Hang et al. proposed a multitask GAN, which uses generator to complete the classification of HSI. Zhan et al. [35] proposed a dynamic neighborhood voting mechanism for GAN classification task. On the basis of output, this mechanism is used to further improve the results, which improves the results of HSI classification to a certain extent. In [37], a symmetric GAN (CAGAN) based on cooperative learning and attention mechanism is proposed. In [36], DcCapsGAN, a dual channel capsule network combined with GAN, is used for HIC.

These GAN methods have made promising progress in HSI classification. However, in practice, it can be found that these methods are hardly trained. In different datasets, these GAN-based methods require complex tuning for hyperparameters and network structure, and improper configuration will lead to the decline of network performance. The reason for this phenomenon is that in some cases, the selected loss function cannot well measure the distance between the real hyperspectral samples and the generated samples, and cannot well indicate the training process. For example, using JS divergence as a measure will cause the gradient to disappear at the beginning of the training. Using the least square loss, the initial training speed is very fast, but when the discriminator cannot recognize the samples generated by the generator, the loss tends to zero. In these cases, the classifier cannot classify hyperspectral data well. Especially in the case of small samples, there are many parameters of the neural network, but few labeled hyperspectral data are provided. Improper loss functions will reduce the classification accuracy more.

Overall, we are facing two challenges.

1) How to accurately measure the distance between the generated samples and the real HSI samples in the training process of GAN. Hyperspectral models are highly diverse. The characteristics of HSI obtained by different shooting tools, shooting angles, and surrounding environment are different. The complexity of these characteristics makes it difficult to measure the distance between the generated samples and hyperspectral samples with a single loss function GAN. If the loss function cannot measure the distance between the real HSI samples and the generated samples, the authenticity of the generated samples is poor, and the discriminator will reject these samples with a great probability. In this case, GAN will not be able to play its advantages, resulting in poor classification effect and great fluctuation.

2) How to solve the problem of overfitting of deep neural network with small training data. With the development of hyperspectral spectrometer, the spatial and spectral resolution of HSI are increasing, the images in the same scene will inevitably become large, and the manual labeling of HSI

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

is time-consuming and laborious, hence the proportion of training samples used for HSI classification will be small. In this case, the neural network will appear overfitting phenomenon, which means that the classification accuracy will be high in the training samples and low in the test samples. This is mainly due to too many parameters of the neural network, and small samples are not enough to tune them. In particular, some HSI classes are imbalance distributed, and some classes even have only a few samples, which further aggravates the overfitting problem. A feasible solution to overfitting is to reduce the size and parameters of neural network.

To tame the above-mentioned challenges, in this article, we design a new immune evolutionary generative adversarial network (HIEGAN) for HIC. In HIEGAN, the structure of generator and discriminator based on ghost [40] module is designed, which has fewer parameters than the convolutional structure. HIEGAN designed multiple metrics (GAN loss function) for the generator. During each adversarial iteration, these loss functions generate different generators through backpropagation, and the discriminator is used as the environment to evaluate the generator performance. Through a certain evaluation method, the optimal individual of these generators is selected to enter the next iteration process. In the meantime, the optimal generator individual is retained in the whole training process. When the generator performance degrades, the reserved optimal individual is used to enter the next iteration. These changes transform the training process into a process of immune optimization using different metrics. In each training process, the optimal loss function will be selected, which makes HIEGAN use the best measurement method when dealing with different HSIs. HIEGAN greatly improves the performance of the generator, reduces the instability during GAN training, and avoids the phenomenon of gradient disappearance and mode collapse caused by insufficient stability.

The main contributions of this article are summarized as follows.

1) We propose an end-to-end HSI classification framework, HIEGAN. HIEGAN preserves the optimal network parameter set in the training process to avoid the degradation of network performance, which reduces the fluctuation of the final classification results and improves the stability of training.
2) HIEGAN uses multiple metrics to jointly optimize the generator, and the discriminator determines which one is used during each iteration. For different HSI samples, HIEGAN can select the most appropriate metrics to measure the distance between the generated sample and the real HSI sample, which fully utilizes the multiple GAN's advantages and improves the quality of HSI samples from the generator.
3) We design a HIEGAN structure based on ghost module, which alleviates the overfitting problem by reducing the number of network parameters. Small sample experiments on three widely used hyperspectral datasets demonstrate the effectiveness of the proposed structure.

The rest of this article is organized as follows. In Section II, we present background knowledge of GANs and evolutionary GAN (EGAN). In Section III, we describe the proposed method in detail. Section IV validates the proposed method on three real datasets and analyzes the hyperparameters in the proposed method. Finally, Section V presents the concluding remarks of this article.

II. Related Work

Before introducing the proposed method, we first review the two important foundations of this article, GAN and EGAN.

A. Generative Adversarial Networks

The GANs was proposed by Goodfellow [41] in 2014. After several years of development, it has become one of the most important branches in the field of deep learning. As shown in Fig. 1, the structure of GAN includes two modules, generator module G and discriminator module D, which are usually composed of neural networks. The generator receives the low-dimensional random vector and produces the data with the same size as the real data sample. The discriminator receives the input real data and the fake data generated by G at the same time, distinguishes their true and false, and outputs the judgment. The training process of GAN is also a process of Nash game. G tries to learn the distribution of real data, transforms the input random variables into data that obeys the real distribution, and D tries to determine whether the input data comes from real samples as much as possible, so as to learn the ability to distinguish the true from the false.

The training process of GAN can be understood as a minimax game process, and its objective function is

$$\min_G \max_D V(D,G) = E_{x\sim p(x)}\left[\log(D(x))\right] + E_{Z\sim p(Z)[\log(1-D(G(Z)))]} \quad (1)$$

where Z is the input of the generator, which is a fixed length random variable with a prior distribution $p(Z)$, and is set as Gaussian distribution in most cases. The real data input is represented by x, and $p(x)$ is the inherent distribution of the real data. After generator, the random variable becomes $G(Z)$, which is pseudo data of the same size as x. When the real data x and the generated sample $G(Z)$ are input to the discriminator, $D(x)$ and$D(G(Z))$ are output from the discriminator, respectively, which is "real/fake" in Fig. 1. Log and E, respectively, represent the logarithm and mean value of discriminator results, which is a common operation of neural network loss function. In the training process, the parameters of the discriminator and generator are updated in turn by backpropagation from the gradient of the loss function, trying to maximize and minimize the loss function $V(D,G)$, respectively. When G is trained, the parameters of D are fixed and $V(D,G)$ is minimized to update the parameters of G. When D is trained, the parameters of G are fixed and $V(D,G)$ is maximized to update the parameters of D.

The process of GAN training is also the process of looking for Nash equilibrium. In the most ideal result, the generator can capture the distribution from the real data, that is, $p(Z) = p(x)$. The generator maps the random variable Z to a new sample $G(Z)$ which is very close to the real data. At this time, the discriminator cannot judge whether the input sample is real or generated by the generator. But, in practice, the training

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

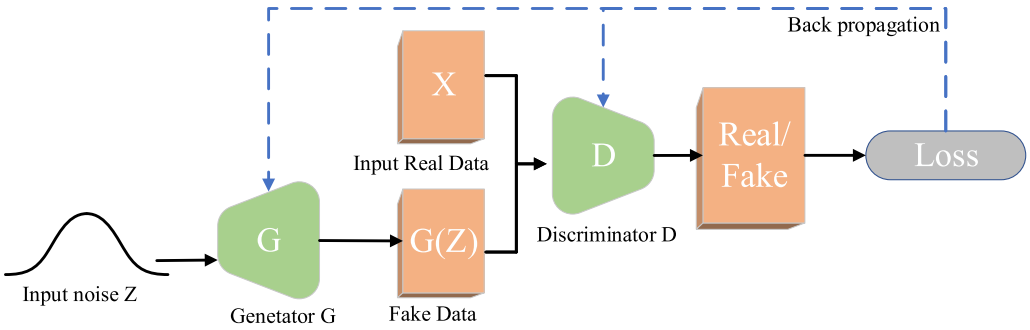


Fig. 1. Framework of GAN.

process of GAN is very difficult to control. For different tasks, we need to repeatedly select the appropriate hyperparameters and network structure.

B. Evolutionary GANs

The concept of EGANs [42] was proposed by Wang et al. in 2018. It combines the idea of evolutionary algorithm with GAN to improve the stability and generation performance of GAN. Inspired by the biological evolution of nature, the evolutionary strategy simulates the problem itself by population, and obtains the optimal solution of the problem through genetic variation among individuals and selection of environment. In EGAN, multiple generators are used as population and discriminator is used as environment. Each individual in the population is a generator. They have the same network structure but different parameters. In the process of evolution, the generator individuals produce different offspring through genetic variation, and enter into a new iteration for further training. In detail, each evolution is divided into three steps.

1) Mutation: In evolutionary algorithm, mutation is the main way to generate new individuals. In EGAN training, first keep the parameters of generator G fixed, update the parameters of D with discriminator loss function, and then fix the parameters of D, set multiple generator loss functions, each generator loss function will generate a set of generator parameters through backpropagation, at this time we can understand that there are multiple generators, which completes the mutation process. Note that throughout the process, although the generator has multiple sets of parameters, there is only one network structure.

2) Evaluation: There are two kinds of evaluation indexes for generated samples: 1) the quality of the generated samples and 2) the diversity of the samples generated. For the former, only the generated samples need to be input into discriminator D and determined according to the average output value of discriminator. For the latter, the gradient value of the discriminator after the image is input to the discriminator is used to evaluate. A smaller discriminator gradient corresponds to a larger sample distribution. Adding these two evaluation indexes in a certain proportion can be used as the evaluation criteria.

3) Selection: Choose the best individual generator or two to enter the next generation iteration.

It is very easy to train the discriminator in the GAN. When training the same epochs, the discriminator will have better performance than the generator. The generator in the EGAN chooses the best one among many in each iteration, so it can have better performance and is more conducive to the balance and stability of GAN. When selecting multiple generators, the diversity of the generated samples is considered, which avoid the pattern collapse problem caused by the collapse of the generated samples in GAN.

III. Method

In this part, the proposed method of HIEGANs will be described in detail. The framework of HIEGAN is shown in Fig. 2, which mainly includes two parts based on the ghost module: discriminator and generator. On the basis of the generator network structure, we add the process of mutation and evaluation, and the memory cells used to preserve the optimal individual, which are introduced in detail below.

A. Ghost Module

Ghost module [40] is a substitute of convolution or deconvolution layer. In the process of convolution, there are many convolution kernels with little difference and the image generated by convolution is also similar. The task of ghost module is to simplify the convolution process. Specifically, ghost transforms the convolution process into the superposition process of a small part of convolution and linear operation. In this way, the parameters of neural network are reduced, the overfitting problem is alleviated, and the amount of calculation is reduced on the basis of fully extracting image features. Specifically, in ghost module, convolution operation transforms the length and width of data and the number of channels, while linear transformation operation only changes the number of channels of data. For example, as shown by the dotted line in Fig. 3, the second ghost operation of the discriminator is enlarged. For the module with input data size of (32,32,64), after the convolution of 64 filters with the size of (3,3), the data size becomes (16,16,64), and then the data size becomes (16,16,128) through linear operation. In the dotted line part of Fig. 4, the second ghost operation of the generator is enlarged. For the module with input data size of (4,4,256) is transformed into data of size (8,8,64) through deconvolution of 64 filters, and then into data

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

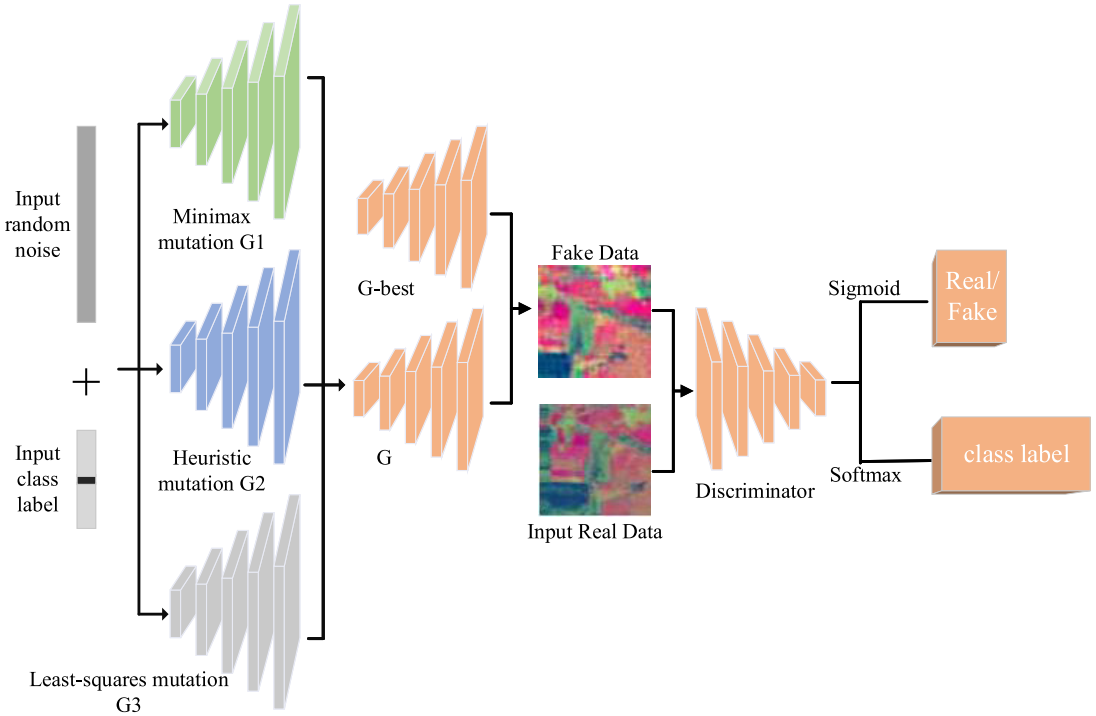


Fig. 2. Framework of HIEGAN.

of size (8,8,128) through linear operation. In ghost module, the stride of convolution and deconvolution is 2, and pooling is not used.

Because of the graphics processing unit (GPU) used for calculation, we use the depth-wise separable convolution as linear operations. We use d to express the ratio of the number of channels generated by the whole ghost module to that generated by ordinary convolution. When d is 1, it means that the whole ghost module is ordinary convolution. When $d = 2$, it means that convolution generates half of the number of image channels, and depth separable convolution generates the other half. The larger the hyperparameter d, the less convolution operations are used and the more deep separable layers are used. In the experiment, we will discuss the influence of d on parameter quantity and classification accuracy.

B. Structure and Loss Function of Discriminator

The function of discriminator in HIEGAN is to judge and classify the input HSI. HSIs need to be preprocessed. The spectral information of each sample in HSI is 1-D vector, which is reduced to 3 by PCA. Taking the spectrum as the center, the surrounding 64 * 64 space is taken as the spatial information of the sample. After this operation, the sample size of each classification becomes (64,64,3). At the same time, the labels are encoded with one-hot. After one-hot coding, we add a bit code for marking the authenticity of each sample. The code of the real hyperspectral sample is 1, and the code of the data generated by the generator is 0.

As shown in Fig. 2, the discriminator receives the real data x and the fake data $G(Z)$ is generated by the generator, and after passing through the multilayer neural networks, it produces two outputs: one is to judge the authenticity of the input hyperspectral data, and the other is the category label.

As shown in Fig. 3, the structure of the discriminator includes four ghost modules and a full connection layer. Ghost module is used to extract image features effectively. Each ghost module uses leaky rectified linear unit (RELU) [43] function as the activation function. Finally, through a fully connected layer, the number of neurons becomes the number of HSI categories plus one, that is, "n_class + 1". Then, n_class neurons are operated by softmax, the cross entropy loss is calculated, and the class is divided. The last neuron is activated by sigmoid for authenticity judgment. The closer the value after sigmoid is to 1, the more likely the image is a real image. Inspired by [44], batchnormal is used between ghost layers to reduce the offset of parameter values between each layer. In Table I, the network structure of the discriminator is given in detail.

The loss function of the discriminator consists of two parts: true false loss and category loss. The specific loss function is

$$\begin{aligned} L_D = & -E_{x\sim p(x)}\left[\log(D(x))\right] \\ & - E_{Z,y\sim p(Z)}\left[\log(1 - D(G(Z, y)))\right] \\ & + E_{x,y\sim p(x,y)}\left[\log p_x(y = D(x, y))\right]. \end{aligned} \tag{2}$$

The formula consists of three terms, in which the authenticity loss of real data x after passing through the discriminator is the first term. When the random vector Z and label y are input, the authenticity loss of fake data $G(Z, y)$ generated by the generator is the second term. The third is the category loss of real data. The first two terms are the same as Formula 1, but each term is preceded by a negative sign. Therefore, the formula is minimized during training to optimize the discriminator.

C. Structure of Generator

In HIEGAN, the goal of generator is to learn the distribution of real data, and make the generated data have the same

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

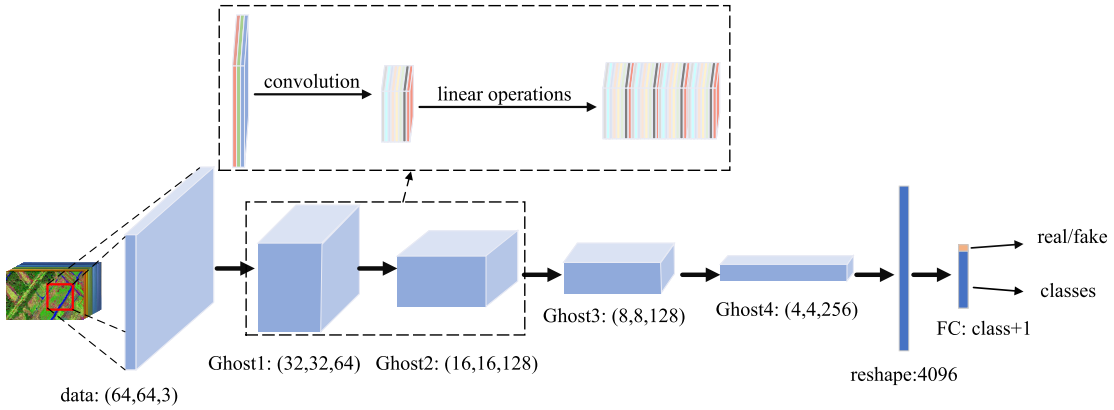


Fig. 3. Structure of discriminator.

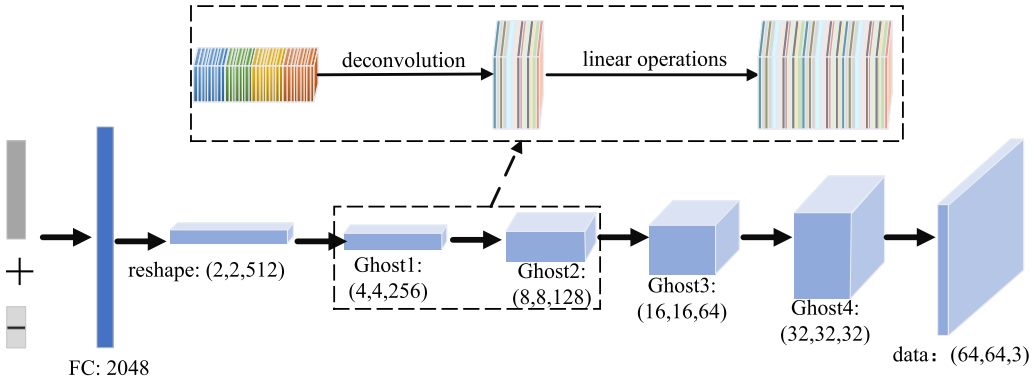


Fig. 4. Structure of generator.

distribution as real data. As shown in Fig. 2, the Gaussian random noise Z and the category one-hot label are connected as the input of generator G, and the fake data with the size of (64,64,3) is generated after multilayer neural network.

The structure of the generator is shown in Fig. 4, including a full connection layer, four ghost modules, and a deconvolution layer. Because the number of categories of each HSI is different, their one-hot vector shape is different, so the first layer is full connect layer, which can transform them into the same format. Four ghost modules and the final deconvolution layer map the low-dimensional data to the data with the same size as the real hyperspectral sample. This not only completes the basic functions, but also reduces the amount of computation. All ghost modules use RELUs as the nonlinear activation function. Batchnormal is added to every layer of the generator. In Table I, the network structure of the generator is given in detail.

In Fig. 2, we can find that there are multiple generators, all of which have the same network structure, but each has different parameters. The following describes the generation process of these generators.

TABLE I
NETWORK OF THE HIEGAN

Nets	Layer	output	stride	Padding	BN?	Activation
G	Fc	256	-	-	Yes	Relu
	ghost.1	(4,4,256)	2	SAME	Yes	Relu
	ghost.2	(8,8,128)	2	SAME	Yes	Relu
	ghost.3	(16,16,128)	2	SAME	Yes	Relu
	ghost.4	(32,32,64)	2	SAME	Yes	Relu
	Dconv	(64,64,3)	2	SAME	-	-
D	ghost.1	(32,32,64)	2	SAME	Yes	LRelu
	ghost.2	(16,16,128)	2	SAME	Yes	LRelu
	ghost.3	(8,8,128)	2	SAME	Yes	LRelu
	ghost.4	(4,4,256)	2	SAME	Yes	LRelu
	Fc	n_ classes+1	-	-	No	Relu
	sigmoid	1	-	-	No	Sigmoid
	softmax	n_ classes	-	-	No	softmax

D. Mutation of Generator

In the training process of GAN, the performance of discriminator is much better than generator, which is not conducive to classification. In order to balance the GAN, HIEGAN adds the evolutionary strategy to mutate and evaluate the generator. Specifically, in each iteration of HIEGAN, the errors of multiple loss functions are calculated, and then these errors are backpropagated to generate multiple sets of generator parameters, and the most appropriate one is selected to enter the next iteration. The process of generating multiple generators from multiple loss functions is also called mutation in evolutionary algorithms. In Fig. 2, generators G1, G2, and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

G3 are generated by backpropagation of minimax objective function, heuristic objective function, and least-squares objective function, respectively.

First, we introduce the class loss part of the generator. Because labels are added to the input of the generator, the samples generated by the generator will have categories. The loss function of the generator needs category loss, and the cross entropy function is used to calculate the category loss. In this article, y is used to represent the category label, and then the category loss can be expressed as

$$L_C = E_{Z,y\sim p(Z,y)}\big[\log p_Z(y = D(G(Z, y)))\big]. \tag{3}$$

Next, we will introduce the total generator loss function. The first is the minimax mutation. It uses JS divergence to measure the distance between the generated data and the real data. This distance is easy to understand and analyze, but it is easy to cause the gradient disappearance problem. Its loss function is as follows:

$$L_{G1} = \frac{1}{2}E_{Z\sim p(Z)}\big[\log(1 - D(G(Z, y)))\big] + L_C. \tag{4}$$

The first part is minimax mutation loss, and the second part is category loss. When the training of generator and discriminator is balanced, minimax function can provide the correct optimization direction. However, when the generated sample is very different from the real sample, when $G(Z, y)$ in the formula tends to 0, the discriminator will reject the sample generated by the generator with high confidence, and then the gradient will disappear. The second is heuristic variation, which maximizes the logarithmic probability of discriminator error and formula is

$$L_{G2} = -\frac{1}{2}E_{Z\sim p(Z)}\big[\log(D(G(Z, y)))\big] + L_C. \tag{5}$$

When the discriminator rejects the samples generated by the generator, the heuristic function will not be saturated. Therefore, it provides effective generator parameter weight update and avoids gradient disappearance. However, the heuristic objective function will lead to the fluctuation of image quality and the instability of training. The third is the least square mutation [45], which is derived from the least square method, and its formula is

$$L_{G3} = E_{Z\sim p(Z)}\big[(D(G(Z, y)) - 1)^2\big] + L_C. \tag{6}$$

When the effect of discriminator identifying the generated samples is poor, the least squares loss function can provide a good optimization direction, but when the distance between the generated samples and the real samples is very small, the least squares loss is close to 0 and the gradient disappears.

After calculating the three loss functions (4)–(6), and using them to backpropagation, the corresponding generator is generated, and the mutation process is completed.

E. Evaluation of Generator

After mutation, multiple generators need to be evaluated in a certain way. Evaluation is the evaluation of individual fitness. In GAN, the discriminator can be understood as the environment of evolutionary algorithm, so the samples generated by the generator can be input into the discriminator to observe the average value of the output. The specific function is

$$F_q = E_Z[D(G(Z, y))]. \tag{7}$$

If the F_q value is large, it is considered that the generated sample is close to the real sample, which means that the generator has high fitness. In GAN, the discriminating ability of discriminator is gradually increasing from scratch. If the generated samples gradually tend to a certain part of the multidimensional space, the discriminator will obviously reject this area in the later stage, which marks this area as false data. Therefore, the diversity of generated samples must be considered. Recently, [46] has proposed a regularization method based on gradient to evaluate the diversity of generation. They use the following items:

$$F_d = -\log\big\|\Delta_D - E_x\big[\log(D(x))\big] - E_Z\big[\log(1 - D(G(Z, y)))\big]\big\| \tag{8}$$

where Δ is the gradient of the discriminator. The larger the F_d is, the more abundant the generated samples. By combining formula (7) with formula (8), it can be used as the evaluation function of the generator

$$F = F_q + \lambda F_d \tag{9}$$

where λ is a coefficient. If a scene with great diversity is needed, a large λ is taken. In this article, its value is set to 0.1. F is the final evaluation function. The larger the F value of each iteration, the better the performance of the generator.

As shown in Fig. 2, during each iteration, three generators G1, G2, and G3 will be generated by mutation. They are evaluated by formula (9), and the one with the largest F value is retained as G.

F. Memory Cell

HIEGAN algorithm always maintains a set of optimal generator parameters in each iteration, which is called memory cells. This idea mainly comes from the memory factor in the immune algorithm.

Specifically, at the beginning of training, a generator G-best is randomly initialized, which is to retain the best generator so far. In the next iteration, after mutation and evaluation, a new G is generated, then G-best and G are evaluated. If G is better than G-best, then G-best is updated to G, and G is used to update the discriminator. If G is not better than G-best, then G-best is used to update the discriminator. In Fig. 2, G-best and G with higher evaluation participate in the next iteration.

In this article, we use formula (9) to evaluate G and G-best. In the early stage of training, the parameters of generator and discriminator are basically random numbers. At this time, in order to ensure the progress of network update, we can consider not adding memory cell module. But in the experiment of this article, G-best always exists.

In this way, the degradation of the generator can be avoided, and every step of the generator is to advance in a better direction.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ALGORITHM OF HIEGAN

Input: the training samples x and labels y from C classes, random parameters generators G, discriminator D, memory cell G-best, the updating times g of the generators and d of the discriminator of every epoch.

```
for every epoch do
    random sample noises Z
    concatenate Z with the labels y
    generate fake data G(Z, y)
    for d steps do
        compute L_D, using formula (2)
        update D by minimizing L_D
    end for
    for g steps do
        compute L_G1, L_G2, L_G3, using formula (4)(5)(6)
        update the parameters of the generators, produce G1, G2, G3
        generate fake data G1(Z, y), G2(Z, y), G3(Z, y), G-best (Z, y)
        compute F_G1, F_G2, F_G3, , F_G-best, using formula(9)
        G ← generator with the highest fitness
        G-best← G
    end for
end for
```

G. HIEGAN Algorithm

After describing the components of HIEGAN, the detailed steps of the algorithm are given in Table II. HIEGAN uses multiple loss functions to enable the generator to evolve, which can combine the benefits of multiple GAN to alleviate the problem of mode collapse and gradient disappearance. With the addition of memory cell, HIEGAN can adaptively maintain the optimal model every time to prevent the degradation of the optimization process. At the same time, ghost module is used in the generator and discriminator of HIEGAN is to ensure high performance and reduce the computational complexity. In this article, experiments are done to verify the effectiveness of this method.

IV. EXPERIMENTAL

In this part, we do experiments on three classic datasets, analyze the results, and discuss them.

A. Experimental Data Description

The first dataset is Indian Pines (IP), which is one of the earliest hyperspectral datasets. The airborne visible infrared imaging spectrometer (AVIRIS) imaged an IP tree in Indiana in 1992. IP have 200 effective bands in the wavelength range of 0.4–2.5 μm, and the size is 145 * 145. The false-color composite image is shown in Fig. 5(a). Apart from the background, IP has 16 categories, and the distribution of samples is very uneven. As shown in Table III, we randomly take 5% of the data of each category as the training data, and the rest as the test data.

The second dataset is Pavia University (PU), which is part of the hyperspectral data of the image of Pavia city in Italy in 2003 by German reflective optics spectral imaging system (ROSIS-03). Pavia university has 103 effective wavebands in the wavelength range of 0.43–0.86 μm, with the size of 610 * 340. The false-color composite image is shown in Fig. 6(a). Apart from the background elements, PU contains nine categories. As shown in Table IV, we randomly selected 2% of each category as training data and the rest as test data.

The last dataset is Salinas (SA), which is an image of SA Valley in California by AVIRIS. The false-color composite image is shown in Fig. 7(a). Apart from the background elements, SA has 16 categories, which are evenly distributed. As shown in Table V, we randomly selected 1% of each category as training data and the rest as test data.

TABLE III
IP CATEGORY DIRECTORY AND NUMBERS OF TRAINING AND TESTING SAMPLES

No	Class	Trainging	Test
1	Alfalfa	2	44
2	Corn-N	71	1357
3	Corn-M	41	789
4	Corn	12	225
5	Grass-M	24	459
6	Grass-T	37	693
7	Grass-P-M	1	27
8	Hay-W	24	454
9	Oats	1	19
10	Soybean-N	49	923
11	Soybean-M	123	2332
12	Soybean-C	30	563
13	Wheat	10	195
14	Woods	63	1202
15	Buildings-G-T-D	19	367
16	Stone-S-T	5	88
Total		512	9737

TABLE IV
PU CATEGORY DIRECTORY AND NUMBERS OF TRAINING AND TESTING SAMPLES

No	Class	Trainging	Test
1	Asphalt	132	6499
2	Meadows	373	18276
3	Gravel	42	2057
4	Trees	61	3003
5	Metal sheets	27	1318
6	Bare soil	100	4929
7	Bitumen	27	1303
8	Bricks	74	3608
9	Shadows	19	928
Total		855	41921

B. Experimental Setting

Our experiment is carried out by an NVIDIA 1660 SUPER GPU with 6 GB display card. We write code through Python 3 and TensorFlow library. On three standard datasets, we use overall accuracy (OA), average accuracy (AA), and kappa coefficient (kappa) to verify the effectiveness of the method. In the IP, PU, and SA datasets, 5%, 2%, and 1% of the data

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SA CATEGORY DIRECTORY AND NUMBERS OF TRAINING AND TESTING SAMPLES

No	Class	Trainging	Test
1	Weeds_1	20	1989
2	Weeds_2	37	3689
3	Fallow	20	1956
4	Fallow_P	14	1380
5	Fallow_S	27	2651
6	Stubble	39	3920
7	Celery	36	3543
8	Grapes	113	11158
9	Soil	62	6141
10	Corn	33	3245
11	Lettuce_4wk	11	1057
12	Lettuce_5wk	19	1908
13	Lettuce_6wk	9	907
14	Lettuce_7wk	11	1059
15	Vinyard_U	72	7196
16	Vinyard_T	18	1789
Total		541	53588

are randomly selected for training, and the rest of the data are used for testing. This is a real small sample.

We select six representative classical HIC methods SVM [17], EPF [15], CNN [28], CNN-PPF [29], 3DCNN [27], MAPC-DRF [21], and two GAN methods, GANs based on three-dimensional information (3DGAN) [39] and HSGAN [31], as comparative experiments. Each method was run for five times independently, and the average and standard deviation were taken as the experimental results. For CNN, three 1×1 sized kernels are used in the convolutional layers as suggested in [28], and the training datasets are expanded eight times through rotation and symmetry operation. For CNN-PPF, the training samples form pixel pairs as described in [29]. For 3DCNN, three 3-D convolution structures are used as suggested in [27], and the spatial window size of 3-D input is resized to 27×27. For MAPC-DRF, the number of decision trees in each random forest is set to be the best in [21]. For 3DGAN [39], we select 4/5 of the training data as training, 1/5 as verification, and select the generation with the best verification results. For HSGAN [31], all data in the datasets is regarded as unsupervised input. Except for the proportion of training samples, all other structures and parameters are the same as those in the original article.

For the HIEGAN proposed in this article, our network structure is shown in Table I. The number of epochs is 600, the generator and discriminator use Adam optimizer, and the learning rate is 0.0001, batch size is 128, and the dimension of random vector Z is 128.

C. Results and Comparison of HIEGAN

1) *Results of the IP Dataset:* Table VI shows the results of nine algorithms. In Table VI, the first 16 rows correspond to the results of each class, the last three rows are the OA, AA, and kappa results of all classes, and the best in each row is bold in black. It is easy to find that the performance of deep structure method is better than that of nondeep structure method. Among them, HIEGAN has a better performance on the whole, and the performance of HIEGAN is slightly worse than other classification methods in the class with less training samples. We find that CNN and MAPC-DRF models are also effective, because the former uses the method of training sample expansion, while the latter has few parameters and performs well in small samples. Compared with other GAN methods, the overall classification accuracy of HIEGAN is about 3% higher than that of 3DGAN and 23% higher than that of HSGAN. HSGAN is a spectral classifier, and its performance is worse than 3DGAN and HIEGAN which are spectral–spatial classifiers.

Fig. 5 shows the classification visualization map of nine algorithms on the IP dataset. As shown in Fig. 5(b)–(J), SVM, EPF, CNN-PPF, 3DCNN, and HSGAN have misclassified many samples in the central region, especially corn and soybean. Compared with these methods, CNN, MAPC-DRF, 3DGAN, and HIEGAN models have significantly better classification results in these areas. HIEGAN performs better than MAPC-DRF and 3DGAN in boundary positioning of stone area.

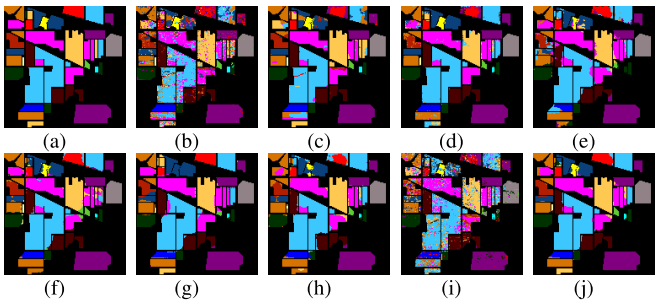


Fig. 5. Classification results on IP. (a) Source Image. (b) SVM. (c) EPF. (d) CNN. (e) CNN-PPF. (f) 3DCNN. (g) MAPC-DRF. (h) 3DGAN. (i) HSGAN. (j) HIEGAN.

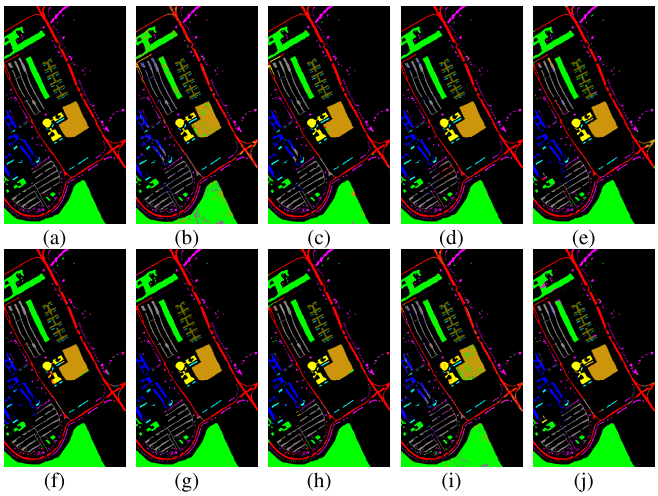


Fig. 6. Classification results on PU. (a) Source Image. (b) SVM. (c) EPF. (d) CNN. (e) CNN-PPF. (f) 3DCNN. (g) MAPC-DRF. (h) 3DGAN. (i) HSGAN. (j) HIEGAN.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CLASSIFICATION RESULTS ON THE IP

Class	SVM	EPF	CNN	CNN-PPF	3DCNN	MAPC-DRF	3DGAN	HSGAN	HIEGAN
1	53.64±14.58	**100.0±0.0**	94.09±2.03	50.90±2.59	88.69±10.69	36.08±26.47	98.18±4.06	34.09±30.36	99.54±1.01
2	69.35±2.93	90.83±1.87	89.49±10.89	87.84±0.34	89.67±1.97	90.82±5.28	89.16±3.36	64.39±2.84	**93.52±2.81**
3	59.64±4.27	89.83±7.24	96.78±1.06	76.98±1.25	86.55±1.79	94.12±4.69	98.88±1.09	58.93±3.28	**97.38±1.68**
4	43.26±2.59	63.87±5.75	83.55±17.53	87.82±2.48	87.17±12.61	92.91±5.76	92.26±6.63	41.60±5.44	**98.75±1.34**
5	86.64±2.52	99.44±0.74	96.90±0.18	92.76±0.41	87.37±2.52	92.54±4.34	94.20±3.43	86.10±4.21	**95.59±2.69**
6	92.30±1.28	98.43±0.82	**99.48±0.31**	92.58±1.06	94.41±2.07	96.49±1.08	94.89±2.60	93.24±1.70	95.64±3.54
7	70.50±11.15	**100.0±0.0**	72.59±6.72	0±0	87.85±10.89	67.14±13.69	42.96±15.84	69.62±9.58	85.92±8.84
8	98.52±0.73	99.95±0.10	**100.0±0.0**	99.29±0.18	97.19±3.18	99.07±1.26	99.73±0.36	90.96±4.57	99.64±0.45
9	46.72±8.75	**93.33±14.90**	80.0±14.12	0±0	47.00±21.09	72.0±26.59	77.89±32.95	10.52±5.26	82.1±12.67
10	62.19±5.72	71.66±8.98	**91.28±5.41**	86.50±1.07	86.52±3.58	89.23±2.33	78.0±2.33	66.02±6.80	89.64±1.09
11	82.61±3.26	95.80±2.36	97.01±1.57	83.59±0.81	93.89±3.54	98.03±0.94	96.56±2.47	74.06±5.43	**98.60±0.97**
12	57.95±7.19	81.54±14.57	**97.79±2.40**	90.12±1.73	85.43±5.90	92.64±1.95	89.98±4.92	52.11±10.47	92.57±3.03
13	90.37±3.87	**100.0±0.0**	98.35±1.37	97.23±0.58	93.75±2.21	89.46±6.01	93.33±3.40	86.35±9.47	94.05±4.79
14	93.72±1.91	96.55±2.65	99.96±0.04	95.92±0.53	97.78±0.83	**99.13±0.44**	99.35±0.36	94.64±0.60	98.81±0.80
15	57.30±6.07	84.52±8.69	68.61±5.19	78.41±1.20	86.37±1.87	**94.19±4.17**	78.85±9.83	38.85±3.70	86.75±2.49
16	81.80±10.34	84.47±8.27	**99.77±0.50**	97.27±0.62	90.32±5.37	16.98±19.97	69.77±10.58	82.50±9.14	78.63±7.42
OA (%)	74.71±1.65	89.44±1.16	94.68±2.10	87.38±0.26	91.28±1.26	93.8±0.51	92.59±0.88	72.64±1.21	**95.54±0.54**
AA (%)	71.66±1.09	90.64±0.78	91.60±2.98	76.07±0.24	87.50±3.86	82.55±2.49	87.12±4.51	65.25±5.02	**92.95±0.50**
Kappa (%)	71.38±1.84	87.98±1.32	93.92±2.42	85.72±0.29	90.06±1.43	92.91±0.59	91.54±1.02	68.76±1.50	**94.91±0.61**

TABLE VII
CLASSIFICATION RESULTS ON THE PU

Class	SVM	EPF	CNN	CNN-PPF	3DCNN	MAPC-DRF	3DGAN	HSGAN	HIEGAN
1	96.45±0.51	**99.53±0.17**	97.47±0.69	96.67±1.51	94.09±3.10	98.62±0.91	96.72±0.60	88.15±0.78	98.72±0.63
2	96.59±0.50	**99.76±0.11**	99.84±0.13	98.07±0.63	98.12±0.43	99.03±0.99	99.37±0.33	95.72±0.76	99.65±0.21
3	72.26±2.47	94.21±2.98	98.58±1.58	83.43±7.90	77.89±2.71	92.12±7.32	**99.15±0.61**	78.30±1.68	93.48±7.71
4	84.27±4.99	96.36±3.81	92.10±2.69	85.56±1.06	94.28±0.80	93.15±1.17	83.93±3.72	83.76±1.62	**96.63±3.19**
5	97.77±0.59	99.47±0.37	99.83±0.29	99.22±0.98	95.40±4.30	96.52±4.93	98.95±0.60	99.08±0.32	**99.98±0.03**
6	73.66±5.30	91.26±8.74	99.24±1.42	91.57±2.76	99.40±0.60	99.70±0.28	92.39±0.71	79.04±3.10	**99.84±0.14**
7	61.22±4.91	86.25±7.38	97.77±1.53	87.15±1.08	91.50±3.40	**99.14±1.70**	83.32±1.73	85.39±3.26	97.91±1.81
8	82.23±5.79	91.80±2.35	94.39±4.45	91.13±2.06	94.80±1.40	**97.91±1.24**	95.67±1.56	82.12±2.22	90.08±5.28
9	99.83±0.10	99.97±0.05	79.26±12.26	98.23±0.65	94.14±2.32	71.36±7.00	73.29±8.96	**99.93±0.09**	96.46±7.06
OA (%)	88.04±1.53	96.79±1.74	97.79±0.1	94.59±0.39	94.82±0.42	97.5±0.65	96.58±0.33	89.58±0.15	**98.07±0.09**
AA (%)	84.92±1.30	95.4±1.99	95.39±1.01	92.34±0.53	92.09±0.30	94.17±1.27	93.3±0.98	87.94±0.38	**96.97±0.75**
Kappa (%)	84.37±1.89	95.76±2.26	97.07±0.14	92.86±0.5	93.14±0.56	96.69±0.86	95.45±0.43	86.11±0.22	**97.44±0.12**

2) *Results of the PU Dataset:* Table VII and Fig. 6 show the results of nine algorithms. We can get the same conclusion through careful observation that the deep learning method is better than the traditional method. The spectral–spatial classification methods are better than the method using only spectral information (HSGAN), and the CNN with data expansion has a good performance. Of all the methods, HIEGAN has the best effect. Moreover, due to the uniform distribution of data in PU dataset, HIEGAN has a good performance in each category.
3) *Results of the Salina Dataset:* Table VIII and Fig. 7 show the results of nine algorithms on SA dataset. In Fig. 7, we can more intuitively feel that HIEGAN is better than other methods, especially in Vinyard_untrained and Lettuce_romaine, the red and gray parts of the graph. On the SA dataset, we only use 1% of the data, which is really small. It can be seen from Table VIII that almost every category of HIEGAN is not the highest, but the overall OA values are the highest.
4) *Visualization of Classification Effect of HIEGAN:* In order to better understand the classification effect of HIEGAN, this article uses T-distributed stochastic neighbor embedding (T-SNE) [47] method for visualization. T-SNE is a method to reduce high-dimensional data to 2-D or 3-D and visualize it. T-SNE can capture most of the local structure of high-dimensional data and reveal the global structure at the same time. In many classification works, after feature extraction of data, the data features are visualized by T-SNE.
The T-SNE feature visualization of hyperspectral data is shown in Fig. 8. The black label in the image corresponds to Tables III–V. It can be seen that HIEGAN has a good effect on the IP dataset, and clearly divides each category. On the SA dataset, although the categories on the scatter plot are close, they are still clearly separated

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
CLASSIFICATION RESULTS ON THE SA

Class	SVM	EPF	CNN	CNN-PPF	3DCNN	MAPC-DRF	3DGAN	HSGAN	HIEGAN
1	99.80±0.15	**100.0±0.0**	99.98±0.02	98.04±0.80	98.25±2.03	99.74±0.45	97.22±1.43	97.81±0.70	97.20±3.31
2	99.09±0.59	99.94±0.12	99.89±0.11	99.62±0.31	99.49±0.70	**100.0±0.0**	99.83±0.29	99.87±0.07	99.13±1.37
3	92.83±0.35	94.81±0.90	**99.92±0.11**	99.56±0.70	95.48±0.94	99.45±1.22	97.82±2.28	89.52±8.72	99.83±0.31
4	96.97±0.28	98.23±0.08	99.02±0.70	**99.68±0.13**	99.08±1.65	90.98±9.54	98.08±3.46	97.48±2.60	97.55±2.71
5	98.42±0.73	**99.93±0.13**	98.06±1.18	96.65±0.93	98.08±1.02	92.06±8.81	99.51±0.52	92.87±3.54	99.63±0.64
6	99.85±0.28	99.99±0.01	**100.0±0.0**	99.38±1.03	99.81±0.22	98.67±1.96	99.70±0.22	99.47±0.46	98.66±1.98
7	98.88±0.72	99.91±0.11	**99.93±0.03**	99.40±0.26	99.16±0.70	98.07±2.99	99.88±0.03	99.21±0.32	98.93±1.08
8	77.17±4.21	90.66±5.17	94.93±5.69	89.51±0.76	93.18±1.66	**99.35±0.85**	97.07±2.82	85.64±2.87	95.48±5.27
9	99.23±0.37	99.88±0.12	**100.0±0.0**	99.36±0.94	97.94±0.83	**100.0±0.0**	100.0±0.0	99.44±0.92	99.83±0.37
10	81.98±3.89	91.75±1.99	94.75±2.54	87.74±2.51	95.19±5.97	**99.56±0.35**	99.35±0.84	86.24±2.85	98.84±1.33
11	89.43±2.86	96.05±1.89	97.52±1.08	92.96±2.17	96.55±1.45	95.85±4.92	**98.44±2.43**	86.57±3.89	96.51±2.81
12	95.80±1.38	99.83±0.12	**99.95±0.02**	99.49±0.56	94.21±3.01	99.86±0.27	99.31±0.60	98.36±1.44	99.92±0.16
13	95.32±1.52	99.06±1.05	**99.77±0.43**	98.12±0.64	94.49±2.84	99.23±0.93	97.39±2.41	96.94±1.23	97.55±2.01
14	88.39±8.0	98.83±1.66	98.81±0.86	91.29±2.08	94.37±0.51	95.39±7.46	99.11±1.93	92.62±1.61	98.82±2.56
15	58.89±3.64	79.15±3.57	81.25±11.25	72.42±2.21	94.33±4.82	94.07±1.17	95.33±4.79	56.12±4.20	**99.54±0.41**
16	96.70±1.39	99.77±0.27	99.92±0.09	95.28±1.33	93.34±3.29	95.65±3.34	**100.0±0.0**	95.26±4.28	99.07±1.05
OA (%)	86.84±1.15	94.12±1.09	95.89±0.59	92.40±0.76	96.14±1.18	97.83±0.60	98.31±0.74	88.56±1.03	**98.32±1.10**
AA (%)	91.80±0.49	96.74±0.44	97.73±0.31	94.91±0.76	96.44±1.00	97.38±0.82	**98.63±0.72**	92.09±1.27	98.53±0.47
Kappa (%)	85.39±1.28	93.46±1.22	95.42±0.66	91.55±0.86	95.70±1.31	97.59±0.67	98.12±0.82	87.23±1.16	**98.13±1.22**

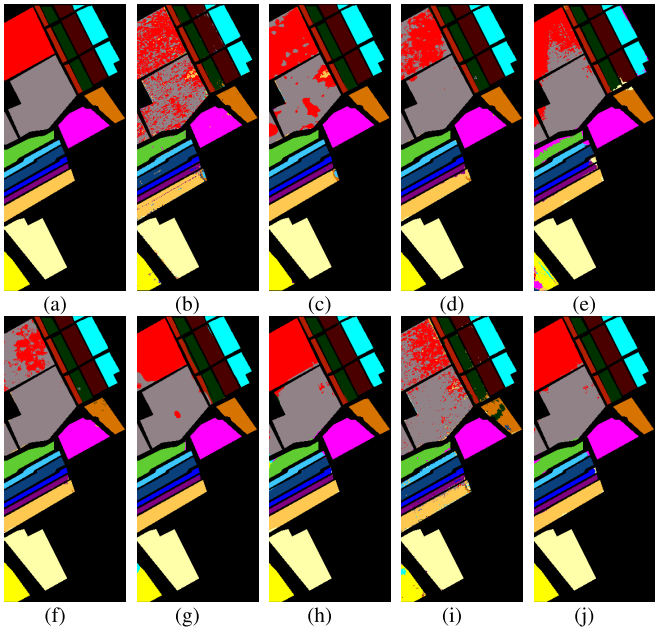


Fig. 7. Classification results on SA. (a) Source Image. (b) SVM. (c) EPF. (d) CNN. (e) CNN-PPF. (f) 3DCNN. (g) MAPC-DRF. (h) 3DGAN. (i) HSGAN. (j) HIEGAN.

and there is almost no overlap. On the dataset of PU, HIEGAN's classification effect is chaotic, but the actual OA value is very high.

D. Classification Results With Fixed Number of Training Samples

In order to test the classification effect of HIEGAN in smaller samples, we add an experiment with fixed number of training samples. On the datasets of IP, PU, and SA, the number of training in each category is fixed to 5, 10, 15, and 20, and the remaining samples are used for testing. Since there are only 20 samples for a category in IP dataset, we select 19 training samples for this experiment. We choose SVM, EPF, CNN, 3DGAN, HSGAN as the comparative experiment.

As shown in Fig. 9, the OA results of the three datasets under different training samples can be seen. When the training samples are so small, the results of HIEGAN are better than other methods. Especially on the IP and SA datasets, each OA value of HIEGAN is the highest. By adding the part of evolution and immunity to GAN, HIEGAN has better robustness to small samples.

TABLE IX
ABLATION EXPERIMENT IP, PU, SA DATASETS

Dataset		D	G1	G2	G3	HIEGAN
IP	OA(%)	94.02	95.12	95.04	94.84	95.54
	AA(%)	87.02	91.72	89.38	90.46	92.95
	Kappa(%)	93.17	94.43	94.35	94.12	94.91
PU	OA(%)	96.22	97.35	97.21	97.15	98.07
	AA(%)	92.47	93.92	94.04	94.60	96.97
	Kappa(%)	94.98	96.48	96.29	96.23	97.44
SA	OA(%)	97.04	97.39	97.82	97.56	98.32
	AA(%)	96.75	97.52	97.97	97.92	98.53
	Kappa(%)	96.70	97.09	97.57	97.28	98.13

E. Ablation Study

In order to study the performance of different objective functions, we extracted four subparts of the method for ablation experiments. The three objective functions of this method are minimax objective function, heuristic objective function, and least-square objective function. We extract these three objective functions to form their own GANs for the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

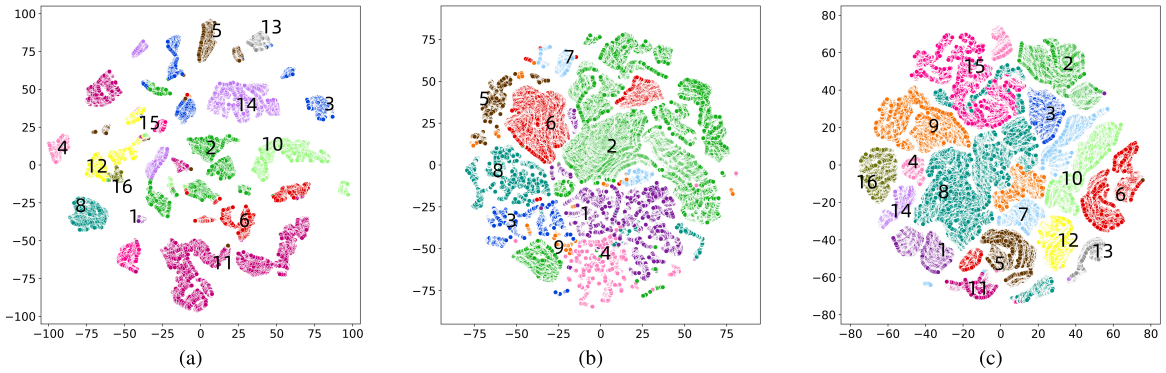


Fig. 8. Visualization of HIEGAN by T-SNE. (a) On IP dataset. (b) On Pavia University dataset. (c) On SA dataset.

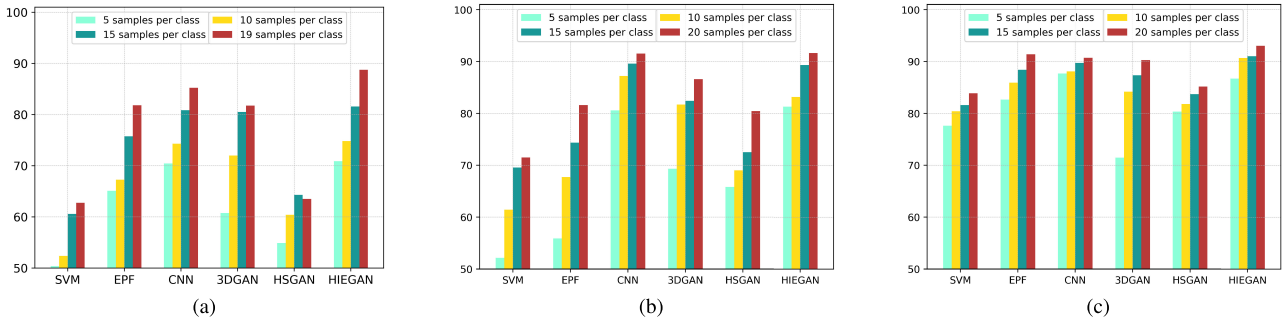


Fig. 9. OA Results With Fixed Number of Training Samples. (a) On IP dataset. (b) On PU dataset. (c) On SA dataset.

classification of hyperspectral data, that is, G1, G2, and G3 in Table IX. In addition, the discriminator itself is an HSI classifier composed of ghost module and we also take it as a part of ablation experiment, namely *D* in Table IX. The experiment was conducted on the IP dataset, PU dataset, and SA dataset, with 5%, 2%, 1% data for each dataset as training data, and the rest as test data. Other experimental parameters are the same as other parts of this article.

In Table IX, we can find that the experimental results of G1, G2, and G3 using the GANs are better than those using only the discriminator, and the OA is about 1% higher. The experimental results of the three objective functions have little difference, and they have their own advantages and disadvantages in different datasets. After using the immune evolution strategy, the experimental results are further improved, with the AA on IP and PU datasets being more than 2% higher than that on G1, G2, and G3. Ablation experiments show that our strategy is effective and immune evolutionary GANs is feasible in hyperspectral data classification.

F. Effects of Different Ghost Parameters on the Results

In ghost module, we use a part of deep separable convolution instead of convolution operation. As shown in Table X, we investigated the influence of different d values on the classification effect and parameter quantity (only considering discriminator). The experiment was carried out on the IP dataset with a training ratio of 5%. It can be seen from Table X that when $d = 1$, that is, when only the convolution layer is used, the accuracy is the highest, and with the increase of d, the accuracy decreases. When $d = 8$, the network parameters are only 141 793, which is very small. In the experiments,

TABLE X
DIFFERENT CLASSIFICATION EFFECT AND PARAMETER QUANTITY WITH DIFFERENT RATE VALUE IN GHOST MODULE

Class	d=1	d=2	d=4	d=8
1	100.00	97.73	93.18	90.91
2	92.23	94.32	91.22	85.62
3	98.85	97.33	94.80	97.97
4	93.77	99.11	99.11	97.78
5	94.33	97.82	97.16	98.47
6	94.80	98.56	95.52	96.68
7	55.56	85.19	33.33	0.0
8	100.0	99.78	100.0	100.0
9	84.21	73.68	73.68	52.63
10	90.79	91.00	90.89	84.07
11	98.16	97.30	98.20	98.16
12	92.36	89.17	90.05	85.44
13	96.41	91.79	94.87	92.31
14	99.75	98.50	98.17	98.67
15	96.73	86.38	88.82	84.47
16	89.77	81.82	76.14	93.18
OA (%)	95.82	95.50	94.78	93.30
AA (%)	92.36	92.46	88.44	84.77
Kappa (%)	95.24	94.86	94.04	92.36
Weights	589201	332881	204721	140641

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

we choose $d = 2$, the parameter is reduced by half, and the accuracy remains high.

V. Conclusion

In this article, we propose an improved GAN structure, HIEGAN, for the task of small sample HIC. The proposed HIEGAN uses ghost module to replace the convolutional layer and deconvolutional layer, which alleviates the overfitting problem by reducing the number of network parameters. In order to accurately measure the distance between the generated sample and HSI sample in different HSIs, HIEGAN sets up multiple generators. One of them exists from the beginning of training to preserve the optimal network parameter set in the training process to avoid the degradation of network performance. The remaining generators are generated by backpropagation of different loss functions in the training process. HIEGAN uses discriminators to evaluate these generators from the two aspects of quality and diversity of generated samples, and selects the best generator to enter the next iteration, which greatly improves the generating capacity of the generator and enhances the robustness of the network. Experimental results show that HIEGAN maintains a strong classification ability in the case of small samples.

References

[1] J. Jia, Y. Wang, J. Chen, R. Guo, R. Shu, and J. Wang, "Status and application of advanced airborne hyperspectral imaging technology: A review," *Infr. Phys. Technol.*, vol. 104, Jan. 2020, Art. no. 103115.

[2] X. Kang, X. Xiang, S. Li, and J. A. Benediktsson, "PCA-based edge-preserving features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7140–7151, Dec. 2017.

[3] P. Ghamisi et al., "New frontiers in spectral-spatial hyperspectral image classification: The latest advances based on mathematical morphology, Markov random fields, segmentation, sparse representation, and deep learning," *IEEE Geosci. Remote Sens. Mag.*, vol. 6, no. 3, pp. 10–43, Sep. 2018.

[4] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral–spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2017.

[5] H. Du, H. Qi, X. Wang, R. Ramanath, and W. E. Snyder, "Band selection using independent component analysis for hyperspectral image processing," in *Proc. 32nd Appl. Imag. Pattern Recognit. Workshop*, Oct. 2003, pp. 93–98.

[6] Y. Zhong and L. Zhang, "An adaptive artificial immune network for supervised classification of multi-/hyperspectral remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 894–909, Mar. 2012.

[7] M. L. Uss, "Maximum likelihood estimation of spatially correlated signal-dependent noise in hyperspectral images," *Opt. Eng.*, vol. 51, no. 11, Jun. 2012, Art. no. 111712.

[8] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4085–4098, Nov. 2010.

[9] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Spectral–spatial hyperspectral image segmentation using subspace multinomial logistic regression and Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 809–823, Mar. 2012.

[10] Y. Chen, N. M. Nasrabadi, and T. D. Tran, "Hyperspectral image classification via kernel sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 1, pp. 217–231, Jan. 2013.

[11] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[12] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

[13] M. Pesaresi and J. A. Benediktsson, "A new approach for the morphological segmentation of high-resolution satellite imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 39, no. 2, pp. 309–320, Feb. 2001.

[14] Y. Yuan, J. Lin, and Q. Wang, "Hyperspectral image classification via multitask joint sparse representation and stepwise MRF optimization," *IEEE Trans. Cybern.*, vol. 46, no. 12, pp. 2966–2977, Dec. 2016.

[15] X. Kang, S. Li, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification with edge-preserving filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 5, pp. 2666–2677, May 2014.

[16] C. Chen, W. Li, H. Su, and K. Liu, "Spectral–spatial classification of hyperspectral image based on kernel extreme learning machine," *Remote Sens.*, vol. 6, no. 6, pp. 5795–5814, 2014.

[17] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[18] Y. Tarabalka, M. Fauvel, J. Chanussot, and J. A. Benediktsson, "SVM- and MRF-based method for accurate classification of hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 7, no. 4, pp. 736–740, Oct. 2010.

[19] M. Fauvel, J. A. Benediktsson, J. Chanussot, and J. R. Sveinsson, "Spectral and spatial classification of hyperspectral data using SVMs and morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 11, pp. 3804–3814, Nov. 2008.

[20] Y. Gu and K. Feng, "Optimized Laplacian SVM with distance metric learning for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 6, no. 3, pp. 1109–1117, Jun. 2013.

[21] B. Liu et al., "Morphological attribute profile cube and deep random forest for small sample classification of hyperspectral image," *IEEE Access*, vol. 8, pp. 117096–117108, 2020.

[22] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[23] T. Li, J. Zhang, and Y. Zhang, "Classification of hyperspectral image based on deep belief networks," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2014, pp. 5132–5136.

[24] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[25] W. Zhao and S. Du, "Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 8, pp. 4544–4554, Aug. 2016.

[26] M. Zhang, W. Li, and Q. Du, "Diverse region-based CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 27, no. 6, pp. 2623–2634, Jun. 2018.

[27] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Jul. 2016.

[28] S. Yu, S. Jia, and C. Xu, "Convolutional neural networks for hyperspectral image classification," *Neurocomputing*, vol. 219, pp. 88–98, Jan. 2017.

[29] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.

[30] V.-E. Neagoe and P. Diaconescu, "CNN hyperspectral image classification using training sample augmentation with generative adversarial networks," in *Proc. 13th Int. Conf. Commun. (COMM)*, Jun. 2020, pp. 515–519.

[31] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2017.

[32] Z. Zhong, J. Li, D. A. Clausi, and A. Wong, "Generative adversarial networks and conditional random fields for hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 50, no. 7, pp. 3318–3329, Jul. 2020.

[33] J. Feng, H. Yu, L. Wang, X. Cao, X. Zhang, and L. Jiao, "Classification of hyperspectral images based on multiclass spatial–spectral generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5329–5343, Aug. 2019.

[34] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geoence Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.

[35] Y. Zhan et al., "Hyperspectral image classification based on generative adversarial networks with feature fusing and dynamic neighborhood voting mechanism," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2019, pp. 811–814.
[36] J. Wang, S. Guo, R. Huang, L. Li, X. Zhang, and L. Jiao, "Dual-channel capsule generation adversarial network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022.
[37] J. Feng et al., "Generative adversarial networks based on collaborative learning and attention mechanism for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 7, p. 1149, Apr. 2020.
[38] M. Zhang, M. Gong, Y. Mao, J. Li, and Y. Wu, "Unsupervised feature extraction in hyperspectral images based on Wasserstein generative adversarial network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 5, pp. 2669–2688, May 2019.
[39] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.
[40] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1580–1589.
[41] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 1–47.
[42] C. Wang, C. Xu, X. Yao, and D. Tao, "Evolutionary generative adversarial networks," *IEEE Trans. Evol. Comput.*, vol. 23, no. 6, pp. 921–934, Dec. 2019.
[43] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 315–323.
[44] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[45] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "Least squares generative adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2794–2802.
[46] V. Nagarajan and J. Z. Kolter, "Gradient descent GAN optimization is locally stable," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
[47] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 1–27, 2008.



Jing Bai (Senior Member, IEEE) received the B.S. degree in electronic and information engineering from Zhengzhou University, Zhengzhou, China, in 2004, and the Ph.D. degree in pattern recognition and intelligent systems from Xidian University, Xi'an, China, in 2009.

She is currently an Associate Professor with Xidian University. Her research interests include image processing, machine learning, and intelligent information processing.



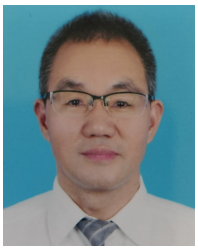
Yang Zhang received the B.S. degree from the School of Artificial Intelligence, Xidian University, Xi'an, China, in 2020, where he is currently pursuing the M.S. degree in computer technology.

He is interested in computer science and intelligent information processing.



Zhu Xiao (Senior Member, IEEE) received the M.S. and Ph.D. degrees in communication and information systems from Xidian University, Xi'an, China, in 2007 and 2009, respectively.

From 2010 to 2012, he was a Research Fellow with the Department of Computer Science and Technology, University of Bedfordshire, Bedfordshire, U.K. He is an Associate Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include wireless localization, the internet of vehicles, and intelligent transportation systems.



Fawang Ye received the Ph.D. degree in computer science from the Beijing Research Institute, Beijing, China, in 2004.

He is currently a Senior Researcher with the National Key Laboratory of Remote Sensing Information and Imagery Analysis, Beijing. His research interests include remote sensing, image processing, deep learning, and intelligent information processing.



You Li received the Ph.D. degree in geographic information science from Wuhan University, Wuhan, China, in 2017.

He is now an Associate Researcher with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China. His main interests include laser radar (LiDAR) processing, 3-D geographic information system (GIS), and 3-D road environment modeling.



Mamoun Alazab (Senior Member, IEEE) received the Ph.D. degree in computer science from the School of Science, Information Technology and Engineering, Federation University Australia, Ballarat, Australia, in 2012.

He is currently an Associate Professor with the College of Engineering, IT and Environment, Charles Darwin University, Casuarina, Australia. He is a Cyber Security Researcher and a Practitioner with the industry and academic experience. He has more than 200 research articles in many international journals and conferences. His research is multidisciplinary that focuses on cyber security and digital forensics of computer systems with a focus on cybercrime detection and prevention.

Dr. Alazab is the Founding Chair of the IEEE Northern Territory (NT) Subsection.



Licheng Jiao (Fellow, IEEE) received the B.S. degree from Shanghai Jiaotong University, Shanghai, China, in 1982, and the M.S. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 1984 and 1990, respectively.

Since 1992, he has been a Professor with the School of Electronic Engineering, Xidian University, Xi'an. He is currently the Director of the Key Laboratory of Intelligent Perception and Image Understanding of the Ministry of Education, International Research Center of Intelligent Perception and Computation, Xi'an. His research interests include intelligent information processing, image processing, machine learning, and pattern recognition.

Prof. Jiao is a Member of the IEEE Xi'an Section Execution Committee, the President of the Computational Intelligence Chapter, the IEEE Xi'an Section, and the Institution of Engineering and Technology (IET) Xi'an Network, the Chairperson of the Awards and Recognition Committee, the Vice Board Chairperson of the Chinese Association of Artificial Intelligence, a Councilor of the Chinese Institute of Electronics, a Committee Member of the Chinese Committee of Neural Networks, and an Expert of the Academic Degrees Committee of the State Council.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:38:56 UTC from IEEE Xplore. Restrictions apply.